

数值计算练习：格点法数值求解Nernst-Planck方程：一维情况

具体情形是解释一个电解水体系（支持电解质为硝酸钾）的各种离子的扩散过程。在该体系中我们先不考虑对流的影响，该过程主要使用Nernst-Planck方程来描述：

$$J_i = -D_i \nabla c_i + z_i D_i \frac{F}{RT} c_i \vec{E}$$

可以看到，离子在电解体系中的扩散主要由三个因素决定，浓度梯度、电场和对流（在我们目前的模型中忽略）。

由法拉第定律我们可以得到：

$$n = \frac{Q}{\nu F}$$

对两侧分别对面积微元和时间求导可以得到：

$$\vec{J} = \frac{\vec{j}}{F}, \text{ 在电解水的体系中我们将 } \nu \text{ 取为 } 1$$

我们代入NP方程可以得到：

$$\vec{j} = F \sum_i z_i \left(-D_i \frac{\partial c_i}{\partial x} + z_i D_i \frac{F}{RT} c_i E \right) = j_{diff} + \kappa E$$

于是：

$$U_{solution} = \int_0^L \frac{j - j_{diff}}{\kappa} dx$$

其中 $U_{solution}$ 是溶液两端的电势差，本模型采用了将这个电势设定为恒定值的假设。这个值不等于电解所用的电压。于是：

$$j = \frac{U_{solution} + \int_0^L \frac{j_{diff}}{\kappa} dx}{\int_0^L \frac{1}{\kappa} dx}$$

于是我们得到了这样的一组关系，我们可以从任意一个时刻的电流密度求出电场，再从电场使用NP方程求出流量，之后由流量求出下一个时刻的浓度，和电流密度，由此可以实现时间递推。

为了实现数值求解这个微分方程，我们将时间和空间均划分为格点，模拟600s的时间，划分为6000个格点，空间网格上划分出100个网格，并划分出网格的面和中心。

我们使用一个四维的向量`np.array([cH,cOH,cK,cNO3])`来描述每一个格点中的离子浓度，随后给每个格点的边界定义通量：

```
import numpy as np
np.set_printoptions(threshold=np.inf, linewidth=200)

def print_state(time_value, cH, cOH, cK, cNO3):
    print(f't = {time_value:.6f} s')
    print("cH =", cH)
```

```

print("cOH =", cOH)
print("cK =", cK)
print("cNO3 =", cNO3)
print()

F=96485.33212
R=8.314462618
Temp=298.15
I = 20e-6
A_liquid = 7e-6
kb=1e-8
U_solution=0.2
L=0.01
N=100
dx=L/N

x_face=np.linspace(0,L,N+1)
x_center= x_face[:-1]+dx/2

T=600
dt=T/6000
t=np.linspace(0,T,6000+1)

cH=np.full(N,1e-4)
cOH=np.full(N,1e-4)
cK=np.full(N,10.0)
cNO3=np.full(N,10.0)

D_H = 9.31e-9
D_OH = 5.27e-9
D_K = 1.96e-9
D_NO3 = 1.90e-9

c=np.array([cH,cOH,cK,cNO3])
pH=-np.log10(cH/1000)
J_H=np.zeros(N+1)
J_OH=np.zeros(N+1)
J_K=np.zeros(N+1)
J_NO3=np.zeros(N+1)

```

这些量会随着时间的进行而更新

之后我们将电极附近看做H和OH的源，并定义每个格点的积分权重：

```

J_source=I/(F*A_liquid)

weight=np.full(N-1,dx)
weight[0]+=dx/2
weight[-1]+=dx/2

```

之后我们开始随着时间格点进行循环，每次循环时使用浓度的平均值去表示格点边上的浓度。使用该浓度计算出电导率，之后算出浓度梯度，并使用梯度算出扩散电流。根据之前定义的积分权重计算两个积分。

```

steps=len(t) - 1
print_state(t[0], CH, COH, CK, CNO3)
for step in range(steps):

    CH_mean=(CH[1:]+CH[:-1])/2
    COH_mean=(COH[1:]+COH[:-1])/2
    CK_mean=(CK[1:]+CK[:-1])/2
    CNO3_mean=(CNO3[1:]+CNO3[:-1])/2
    keppa=(F)**2*(D_H*CH_mean+D_OH*COH_mean+D_K*CK_mean+D_NO3*CNO3_mean)/(R*Temp)

    CH_diff=(CH[1:]-CH[:-1])/dx
    COH_diff=(COH[1:]-COH[:-1])/dx
    CK_diff=(CK[1:]-CK[:-1])/dx
    CNO3_diff=(CNO3[1:]-CNO3[:-1])/dx
    j_diff=-F*(D_H*CH_diff-D_OH*COH_diff+D_K*CK_diff-D_NO3*CNO3_diff)

    integral_diff=np.sum(weight*j_diff/keppa)
    integral_j=np.sum(weight/keppa)

    j=(U_solution+integral_diff)/integral_j
    E=(j-j_diff)/keppa
    I=j*A_liquid

```

将电迁移和浓差迁移带来的流量相加，形成四种离子的流量

```

J_H[1:-1]=-D_H*CH_diff+D_H*(F)*CH_mean*E/(R*Temp)
J_OH[1:-1]=-D_OH*COH_diff-D_OH*(F)*COH_mean*E/(R*Temp)
J_K[1:-1]=-D_K*CK_diff+D_K*(F)*CK_mean*E/(R*Temp)
J_NO3[1:-1]=-D_NO3*CNO3_diff-D_NO3*(F)*CNO3_mean*E/(R*Temp)

J_source=j/F
J_H[0] = J_source
J_OH[-1] = -J_source

```

最后使用流量更新每个格点的浓度。

```

CH_new=CH+dt*(J_H[:-1]-J_H[1:])/dx
COH_new=COH+dt*(J_OH[:-1]-J_OH[1:])/dx
CK_new=CK+dt*(J_K[:-1]-J_K[1:])/dx
CNO3_new=CNO3+dt*(J_NO3[:-1]-J_NO3[1:])/dx

if (np.any(CH_new<=0) or np.any(COH_new<=0) or np.any(CK_new<=0) or
np.any(CNO3_new<=0)):
    break
b=CH_new-COH_new
c_1=(np.abs(b)+np.sqrt(b**2+4*kb))/(2)
c_2=kb/c_1

CH=np.where(b>=0,c_1,c_2)
COH=np.where(b>=0,c_2,c_1)
CK=CK_new
CNO3=CNO3_new

```

```
print_state(t[step + 1], CH, COH, CK, CN03)
```